

calling *mknod* instead of *mkfifo* will reduce the execution time required to create the FIFO file.

The *proc* file system

The *proc* file system is one of the important virtual file systems in Linux. It helps to maintain information about currently running processes, system limitations, and system information (like CPU, RAM, IRQ, etc). The output of the *mount* command lists this line for *proc*:

proc on /proc type proc (rw)

Here, the first *proc* is a pseudo-device created in kernel memory. It is mounted at the */proc* mount-point, with the *proc* file-system type. The size of */proc* is always zero, because the data accessible via */proc* is generated on the fly. For example, if you execute a program (which becomes a running process), then a directory is created in */proc* with the process identification number (PID). The kernel maintains process-related information like its state, PID, PPID, GID, fd table, and other parameters, in the process ID directory, as long as the process is executing in the system. Once the process has ended, the directory is removed. More complete information about */proc* was published in an earlier issue of LFY [9].

Developers can also create their own entries in */proc* to store necessary information about their application programs, by interfacing the *proc* file system with a kernel module. An excellent resource for more information on this is reference [10].

Synchronisation of a shared file

Linux is a multi-processing operating system, and if more than one process wants to access the same file, or the same record in a file, it leads to a race condition. For example, if the first process opens the file, the kernel sets the file pointer to the file. Then, the second process opens the same file. The first pointer is updated with the second pointer, so whatever you modify with the first process will not be reflected in the file; only modifications by the second process will be reflected.

This undesirable situation, caused by the race condition, can be avoided if we synchronise the file with the help of a suitable synchronisation mechanism. Linux supports all kinds of synchronisation mechanisms, at user level as well as kernel level. At the user level, we have semaphores to synchronise a critical section, but to synchronise a file, it may not be a good choice. Instead, file locking is more suitable. In multi-threaded programming, *pthread_mutex* is used to synchronise a thread.

In this section, we discuss file-locking mechanisms: we have two locking mechanisms, which can be implemented by *flock* and *fcntl*. The *fcntl* type of locking is more robust, flexible and fine-grained. Figures 9 through 12 explain the different types of locking architectures. You can get information about the Linux kernel locking mechanisms,

```

File Edit View Terminal Help
[arjun@Arunachaleswara:~/home/arjun]$ mkfifo "myfifo", [{"mkfifo", "myfifo"}, {"50 vars +/}] = 0
[myfifo]
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f807000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
open("/etc/ld.so.cache", O_RDONLY) = 3
.....
mknod("myfifo", S_IFIFO|0666) = 0
close(1) = 0
close(2) = 0
exit_group(0) = 7

```

Figure 7: Running strace mkfifo

% time	seconds	usecs/call	calls	errors	syscall
94.87	0.000462	462	1	0	mknod
5.13	0.000025	2	11	0	mmap2
0.00	0.000000	0	3	0	read
0.00	0.000000	0	5	0	open
0.00	0.000000	0	7	0	close
0.00	0.000000	0	1	0	execve
0.00	0.000000	0	1	0	l access
0.00	0.000000	0	3	0	brk
0.00	0.000000	0	1	0	munmap
0.00	0.000000	0	4	0	mprotect
0.00	0.000000	0	5	0	fstat64
0.00	0.000000	0	1	0	set_thread_area
0.00	0.000000	0	1	0	statfs64
100.00	0.000487		44	1	total

Figure 8: Running strace -c -F mkfifo

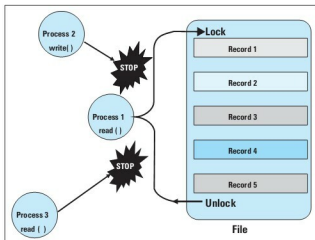


Figure 9: Non-shared lock—Sys V semaphore

and semaphore usage in programming at the user level, from references [10-12].

File-locking implementations: *fcntl*

The *flock* structure is used to declare and initialise a file lock. The structure contains the following members:

```

struct flock {
    short l_type;
    short l_whence;
    off_t l_start;
    off_t l_len;
    pid_t l_pid;
};

```